When AI Misreads Us

Multimodal sentiment analysis and the cost of getting it wrong

SHERRI SON
Fordham University · M.S. Data Science
Advisor: CISC 5800 Faculty

OVERVIEW

Social platforms increasingly use multimodal AI to decide what stays online. When these systems misread sarcasm, visual culture, or mental-health discourse, the cost is real: people get silenced, reporting gets erased, awareness posts get demonetized.

SCALE

1,050
DOCUMENTED CASES OF UNJUSTIFIED CENSORSHIP

60+
COUNTRIES AFFECTED

— Human Rights Watch, 2023

CASE

> A father held his daughter as they died. Instagram took the photo down for nudity. They were both fully clothed.

GAZA · DECEMBER 2023

But who does it get wrong?

The average hides the harm.

Source: Human Rights Watch, 2023

When the algorithm decides what counts as harm.

DATA

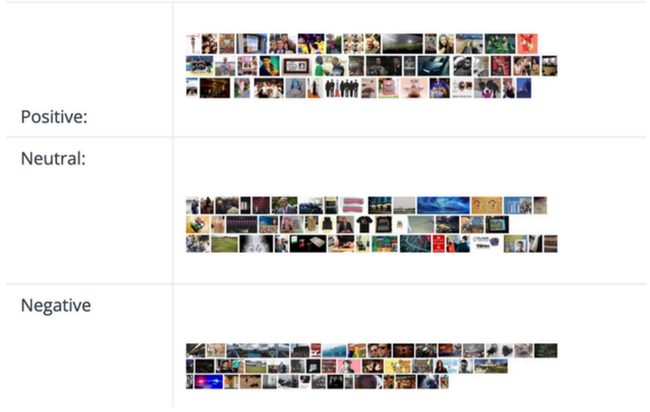
MVSA-Single (Niu et al., 2016)

Def: A set of image-text paris with manual annotations, collected from twitter

Number: 4,511 paired image+text Twitter posts

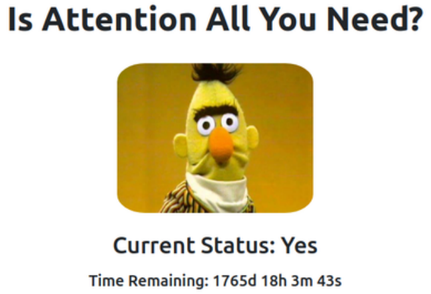
labels: positive · neutral · negative

3,157 / 677 / 677



MODELS

- CLIP zero-shot
- Text SVM (DistilBERT)
- Image SVM (CLIP-ViT-B/32)
- **SVM Fusion**
- **Attention MLP** ← ***focus model***



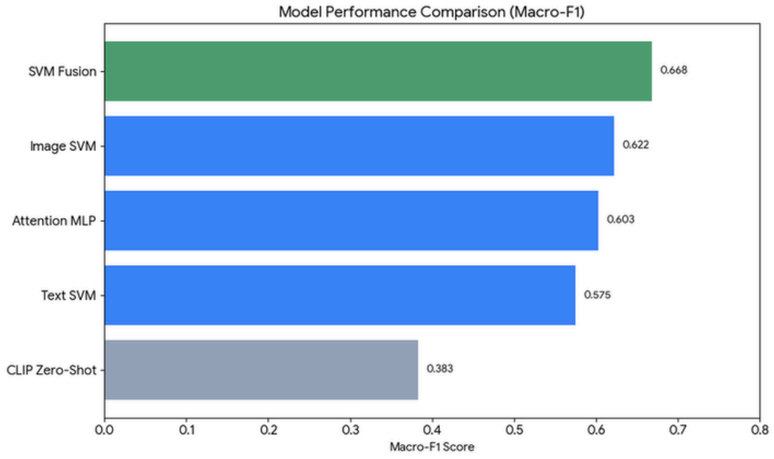

```python
class AttentionFusionMLP(nn.Module):
    """Each modality is projected to a shared space; a learned softmax gate
    decides per post how much to weight text versus image."""
    def __init__(self, txt_dim=768, img_dim=512, proj_dim=256,
                 hidden=128, n_classes=3, dropout=0.3):
        super().__init__()
        self.txt_proj = nn.Linear(txt_dim, proj_dim)
        self.img_proj = nn.Linear(img_dim, proj_dim)
        self.attn = nn.Linear(proj_dim, 1)
        self.head = nn.Sequential(
            nn.Linear(proj_dim, hidden), nn.ReLU(),
            nn.Dropout(dropout), nn.Linear(hidden, n_classes),
        )

    def forward(self, txt, img):
        t = torch.tanh(self.txt_proj(txt))
        i = torch.tanh(self.img_proj(img))
        stacked = torch.stack([t, i], dim=1)          # (B, 2, P)
        weights = F.softmax(self.attn(stacked).squeeze(-1), dim=-1)
        fused   = (stacked * weights.unsqueeze(-1)).sum(dim=1)
        return self.head(fused), weights              # return gates too
```

QUESTION

Surprisngly, our best model was

SVM Fusion

Macro-F1 in here beats the Attention MLP model.



Five models, one dataset, one trap.

PIPELINE

01 · RAW DATA
4,511 paired image+text Twitter posts

↓

02 · TEXT FEATURES
DistilBERT [CLS] embeddings

↓

03 · IMAGE FEATURES
CLIP-ViT-B/32 image encoder

↓

04 · MODELS
5 approaches (zero-shot · SVM × 3 · attention)

↓

05 · EVALUATION
Macro-F1 · partition analysis

Five models, one dataset, one trap.

PIPELINE

A1. Label reconciliation (Niu et al. 2016 protocol)

Drops 358 directly contradictory labels (e.g., text=positive, image=negative). Final dataset: 4,511 posts.

```python
def reconcile(lab):
    parts = [v.strip().lower() for v in str(lab).split(',')]
    if len(parts) != 2:
        return None
    text_lab, img_lab = parts
    valid = {'positive', 'negative', 'neutral'}
    if text_lab not in valid or img_lab not in valid:
        return None
    # Direct conflict (positive vs negative): drop the sample
    if {text_lab, img_lab} == {'positive', 'negative'}:
        return None
    if text_lab == img_lab:
        return text_lab
    # One annotator said neutral: trust the non-neutral one
    if text_lab == 'neutral': return img_lab
    if img_lab == 'neutral':  return text_lab

df['label'] = df['label'].apply(reconcile)
df = df.dropna(subset=['label']).reset_index(drop=True)
print(df['label'].value_counts())
```

```
# positive  2683
# negative  1358
# neutral    470
```

A2. Feature extraction (CLIP + DistilBERT)

Embeddings cached to disk so every downstream model trains on identical features.

```python
# CLIP ViT-B/32 for images (512-dim), DistilBERT for text (768-dim)
clip_model = CLIPModel.from_pretrained("openai/clip-vit-base-patch32").to(device)
bert_model = AutoModel.from_pretrained("distilbert-base-uncased").to(device)

@torch.no_grad()
def extract_clip_images(paths, batch_size=64):
    embs = []
    for i in range(0, len(paths), batch_size):
        imgs = [Image.open(p).convert("RGB") for p in paths[i:i+batch_size]]
        inputs = clip_proc(images=imgs, return_tensors="pt").to(device)
        feats = clip_model.get_image_features(**inputs)
        feats = feats / feats.norm(dim=-1, keepdim=True)   # L2-normalize
        embs.append(feats.cpu().numpy())
    return np.vstack(embs)

img_emb = extract_clip_images(df['image_path'].tolist())  # (N, 512)
txt_emb = extract_distilbert(df['text'].tolist())         # (N, 768)
```

Five models, one dataset, one trap.

PIPELINE

A3. CLIP zero-shot baseline

No training. Cosine similarity between image and three sentiment prompts.

```
# CLIP zero-shot baseline: cosine similarity to sentiment prompts
classes = ['negative', 'neutral', 'positive']
prompts = [f"a photo expressing {c} sentiment" for c in classes]
prompt_embs = encode_clip_text(prompts)          # (3, 512), L2-normalized

sims = test_img_t @ prompt_embs.T                # (677, 3)
preds = sims.argmax(dim=-1).cpu().numpy()

# === CLIP Zero-Shot Results ===
# Accuracy: 0.408
# Macro-F1: 0.383
```

A4. RBF SVMs (text-only, image-only, fusion)

Tuned with PredefinedSplit on the validation set. class_weight='balanced' is mandatory because neutral is only 10% of the data.

```
def fit_svm_with_val(X_tr, X_va, y_tr, y_va, name):
    """Tune RBF SVM on the predefined val split (no CV reshuffle)."""
    X = np.vstack([X_tr, X_va])
    y = np.concatenate([y_tr, y_va])
    fold = np.concatenate([np.full(len(X_tr), -1), np.zeros(len(X_va))])
    ps = PredefinedSplit(fold)
    grid = {'C': [0.1, 1, 10, 100], 'gamma': ['scale', 0.001, 0.01, 0.1]}
    svm = SVC(kernel='rbf', class_weight='balanced')   # neutral is only 10%
    gs = GridSearchCV(svm, grid, cv=ps, scoring='f1_macro', n_jobs=-1)
    gs.fit(X, y)
    return gs.best_estimator_

text_svm   = fit_svm_with_val(train_txt_s,  val_txt_s,  ...)   # 768-dim
img_svm    = fit_svm_with_val(train_img_s,  val_img_s,  ...)   # 512-dim
svm_fusion = fit_svm_with_val(train_concat, val_concat, ...)   # 1280-dim
```

A5. Attention-gated MLP

The forward pass returns the gating weights so we can later inspect, per post, how much the model trusted text versus image.

Five models, one dataset, one trap.

PIPELINE

A6. Partition analysis (the headline finding)

Splits the test set by whether the unimodal SVMs agreed. Every model gets evaluated separately on each subset.

```
# Partition the test set by whether the unimodal models agreed
text_preds = text_svm.predict(test_txt_s)
img_preds  = img_svm.predict(test_img_s)
agree    = (text_preds == img_preds)         # 427 cases (63.1%)
disagree = ~agree                             # 250 cases (36.9%)

# Evaluate every model on each subset separately
for name, p in models.items():
    overall  = f1_score(test_lab,           p,           average='macro')
    f1_agr   = f1_score(test_lab[agree],    p[agree],    average='macro')
    f1_dis   = f1_score(test_lab[disagree], p[disagree], average='macro')
    print(f"{name:18s}  overall={overall:.3f}  "
          f"agree={f1_agr:.3f}  disagree={f1_dis:.3f}")
```

A7. Case taxonomy (10 behavioral types)

Tags each test case with up to one behavioral pattern. Used to populate the case gallery.

```
# 10 behavioral case types in priority order, grouped by what they reveal
case_types = [
    # SAVE: where multimodal fusion adds value
    {'tag': 'Text-driven save',
     'mask': correct['Attention MLP'] & ~correct['SVM fusion'] & (attn[:, 0] > 0.6)},
    {'tag': 'Image-driven save',
     'mask': correct['Attention MLP'] & ~correct['SVM fusion'] & (attn[:, 1] > 0.6)},
    {'tag': 'Modality conflict resolved',
     'mask': (preds['Text SVM'] != preds['Image SVM']) & correct['Attention MLP']},

    # HARM: where fusion makes things worse
    {'tag': 'SVM fusion breaks signal',
     'mask': correct['Text SVM'] & correct['Image SVM'] & ~correct['SVM fusion']},
    {'tag': 'MLP overrides correct SVM',
     'mask': correct['SVM fusion'] & ~correct['Attention MLP']},

    # PATTERN: interesting failure modes
    {'tag': 'Neutral collapse',
     'mask': (y_true == 'neutral') & ~correct['Attention MLP']},

    # BASELINE
    {'tag': 'Universal failure',
     'mask': np.all([~correct[m] for m in preds], axis=0)},
]
```

When the modalities disagree, the gap explodes.

PARTITION FINDING

TEXT SVM

0.757 → 0.310
−44.7 pts

SVM FUSION

0.759 → 0.505
−25.4 pts

ATTN MLP

0.697 → 0.463
−23.4 pts

Agree subset: 427 posts (63.1%)
Disagree subset: 250 posts (36.9%)

agree = (text_preds == img_preds)

WHERE MULTIMODAL FUSION EARNS ITS KEEP

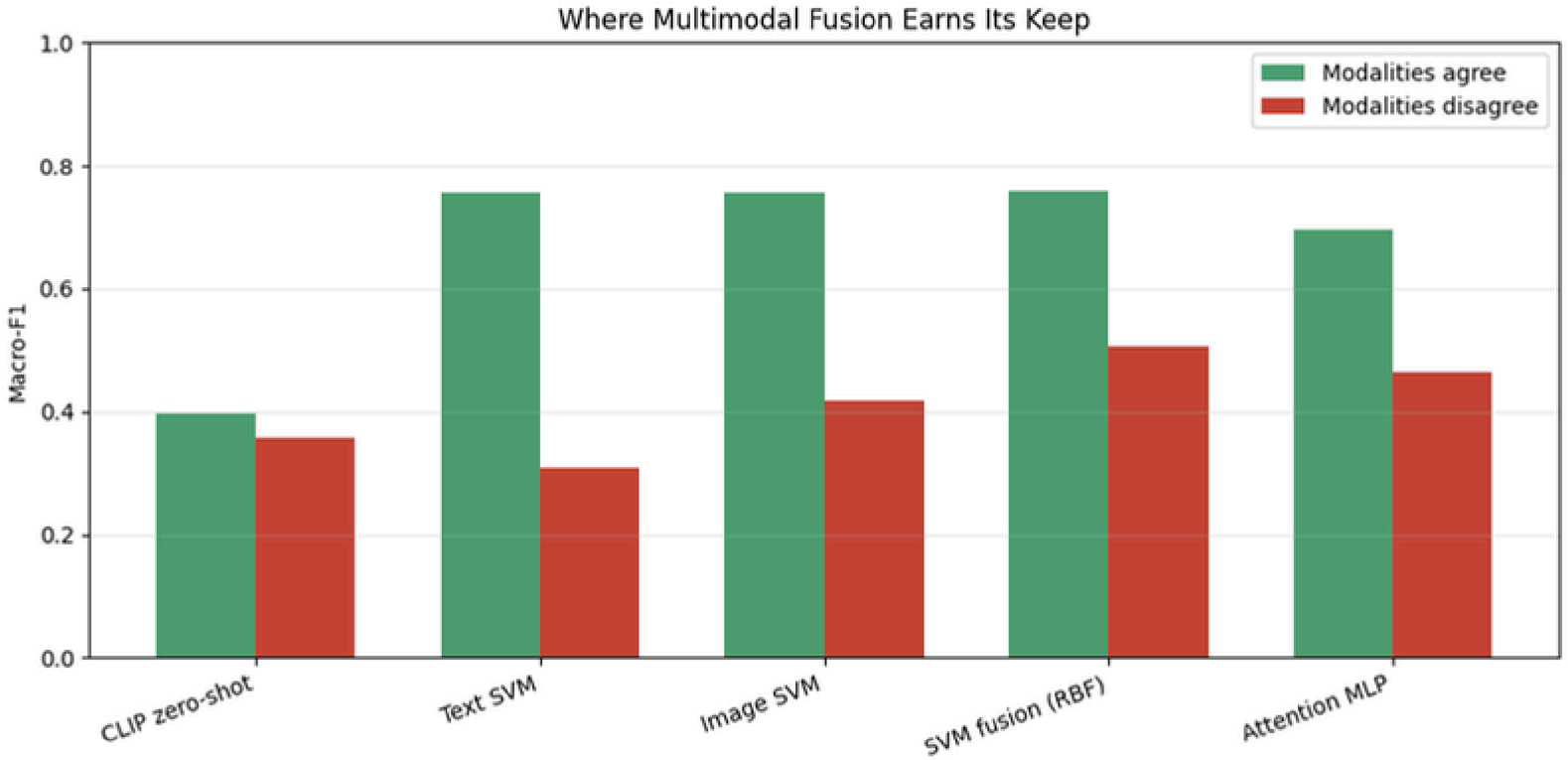


FIG. Where Multimodal Fusion Earns Its Keep — partition by (text_preds == img_preds)

Where it works.
Where it breaks.

CASE STUDIES — REAL POSTS

TEXT-DRIVEN SAVE — SAVE

"RT @hayleystringss: Heartbreaking moment when you realise you've got sparkling water instead of still ??"

true: negative

CLIP zero-shot	✗ neutral
Text SVM	✗ positive
Image SVM	✗ positive
SVM fusion	✗ positive
Attention MLP	✓ negative

attention — T:0.94 I:0.06

UNIVERSAL FAILURE — BASELINE

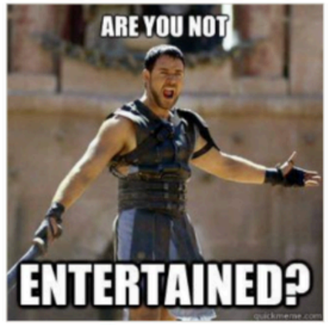
"Christian grey - an emotionally crippled narcissist romanticising domestic abuse as 'erotica', happy valentines day:)"

true: positive

CLIP zero-shot	✗ negative
Text SVM	✗ negative
Image SVM	✗ negative
SVM fusion	✗ negative
Attention MLP	✗ neutral

attention — T:0.33 I:0.67

IMAGE-DRIVEN SAVE — SAVE



"RT @kjo19: Hey Blue Jays Nation..."

true: positive

CLIP zero-shot	✗ neutral
Text SVM	✗ negative
Image SVM	✗ negative
SVM fusion	✗ negative
Attention MLP	✓ positive

attention — T:0.02 I:0.98

NEUTRAL COLLAPSE — PATTERN

"Marca | Casemiro will cut his holidays short and join Real Madrid sooner than expected. #HalaMadrid"

true: neutral

CLIP zero-shot	✓ neutral
Text SVM	✗ negative
Image SVM	✗ positive
SVM fusion	✗ negative
Attention MLP	✗ negative

attention — T:0.75 I:0.25

Stop measuring how often AI is right.
Start measuring who it gets wrong.

Impliments

When accuracy becomes our only metric, AI turns into a tool for digital erasure.

In the real world, "high accuracy" usually means the model is great at the easy stuff, while it systematically silences the most vulnerable—turning a grieving mother's post into "policy-violating" content or a cry for help into an ignored data point because it lacked the "right" keywords.

We must stop measuring how often AI is right and start asking whom it gets wrong, because the "tricky" cases are almost always the human lives that most need to be seen.

DATA & METHODS

Dataset · MVSA-Single (n=4,511)
Split · 70 / 15 / 15 stratified
train 3,157 · val 677 · test 677
Models · 5 (CLIP zero-shot,
Text SVM, Image SVM,
SVM Fusion, Attention MLP)
Evaluation · Macro-F1, partition
analysis (agree vs disagree)

ACKNOWLEDGMENTS

Niu, T. et al. (2016). MVSA: Sentiment analysis on multi-view social data. MultiMedia Modeling, 15–27.
Radford, A. et al. (2021). Learning transferable visual models from natural language supervision. CLIP.
Sanh, V. et al. (2019). DistilBERT, a distilled version of BERT.

Source: Human Rights Watch (2023) report on automated content moderation, 1,050 documented cases across 60+ countries.